C++ in Summary

1. C++ Compiler

- **Compiler**
  - Converting high-level language code to machine language code



- C++ compiler generates executable file (binary code) from C++ source code (files).

- Visual Studio 20xx
  - Installed in our PC rooms (Building 208 4th, 5th, 6th floors)

- g++ (LINUX, macOS, MS Windows, cygwin, …)
  > g++ -o TestProg TestProg.cpp
  ➢ ./TestProg

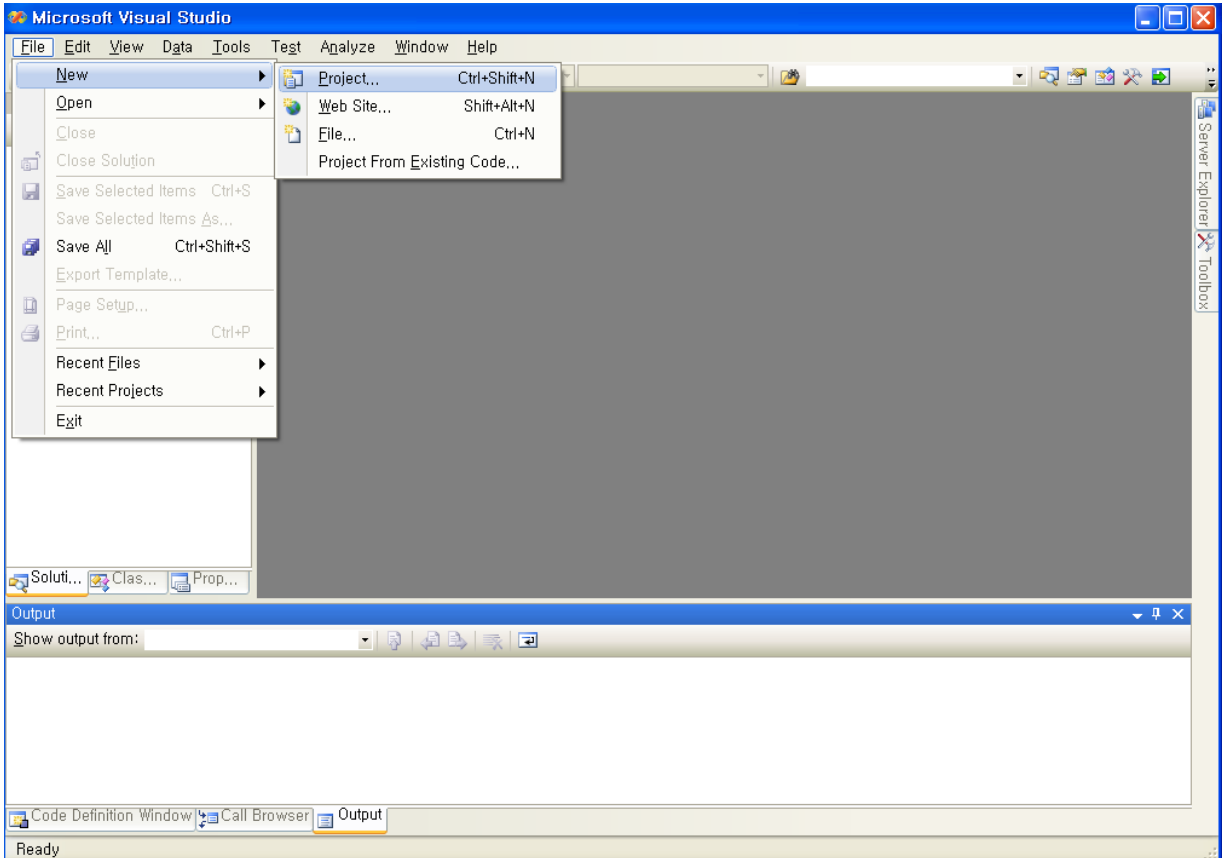
2. Useful Youtube Tutorials for C++ Beginner

- Compile C++ with Visual Studio 2017
  https://youtu.be/2Oi0jynRy9E

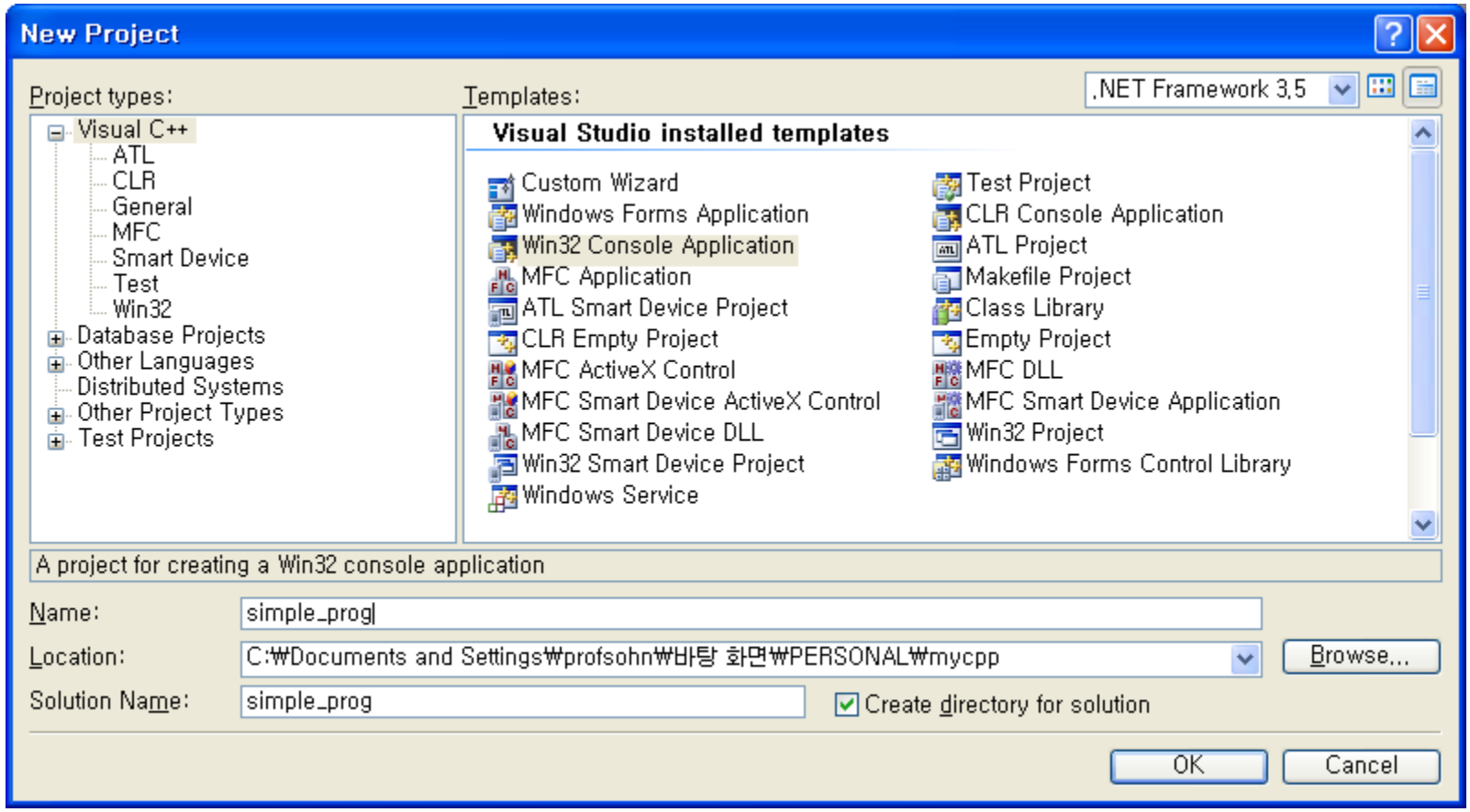
- C++ Tutorial for Beginners – Full Course
  https://youtu.be/vLnPwxZdW4Y

- Using Makefile for compiling multiple C++ source files
  https://youtu.be/aw9wHbFTnAQ

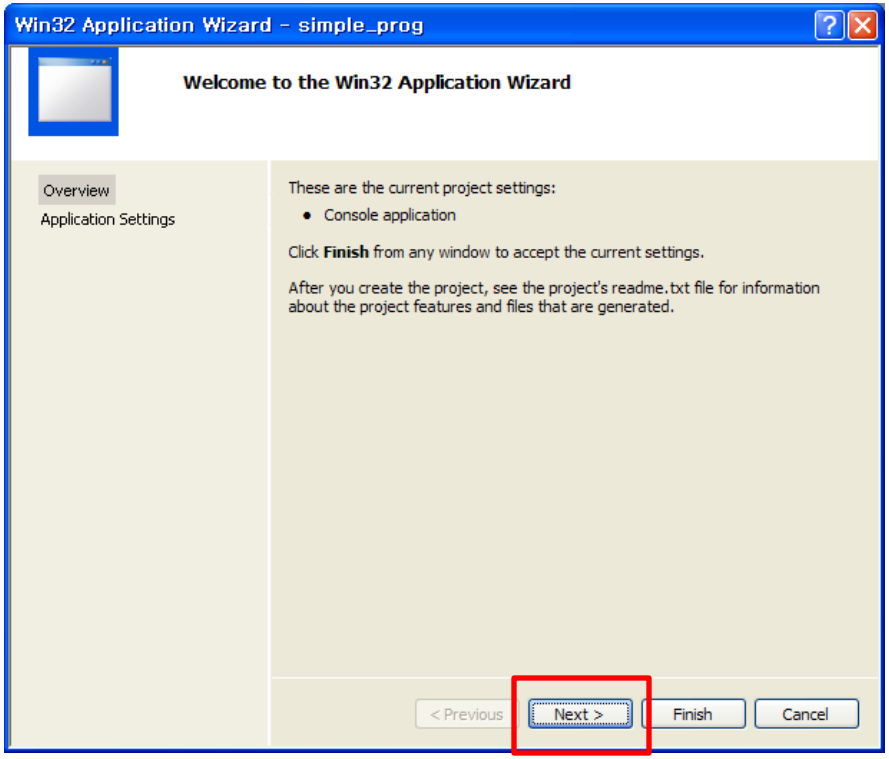
Visual Studio 2008 : Creating Command-Line Program

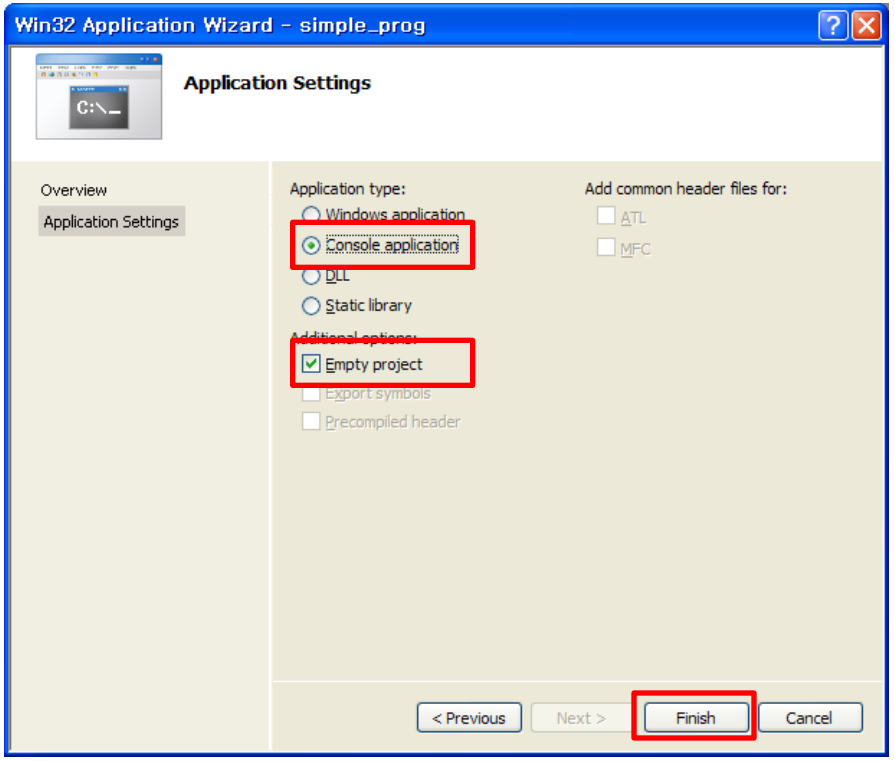


Visual Studio 2008
Creating Command-Line Program

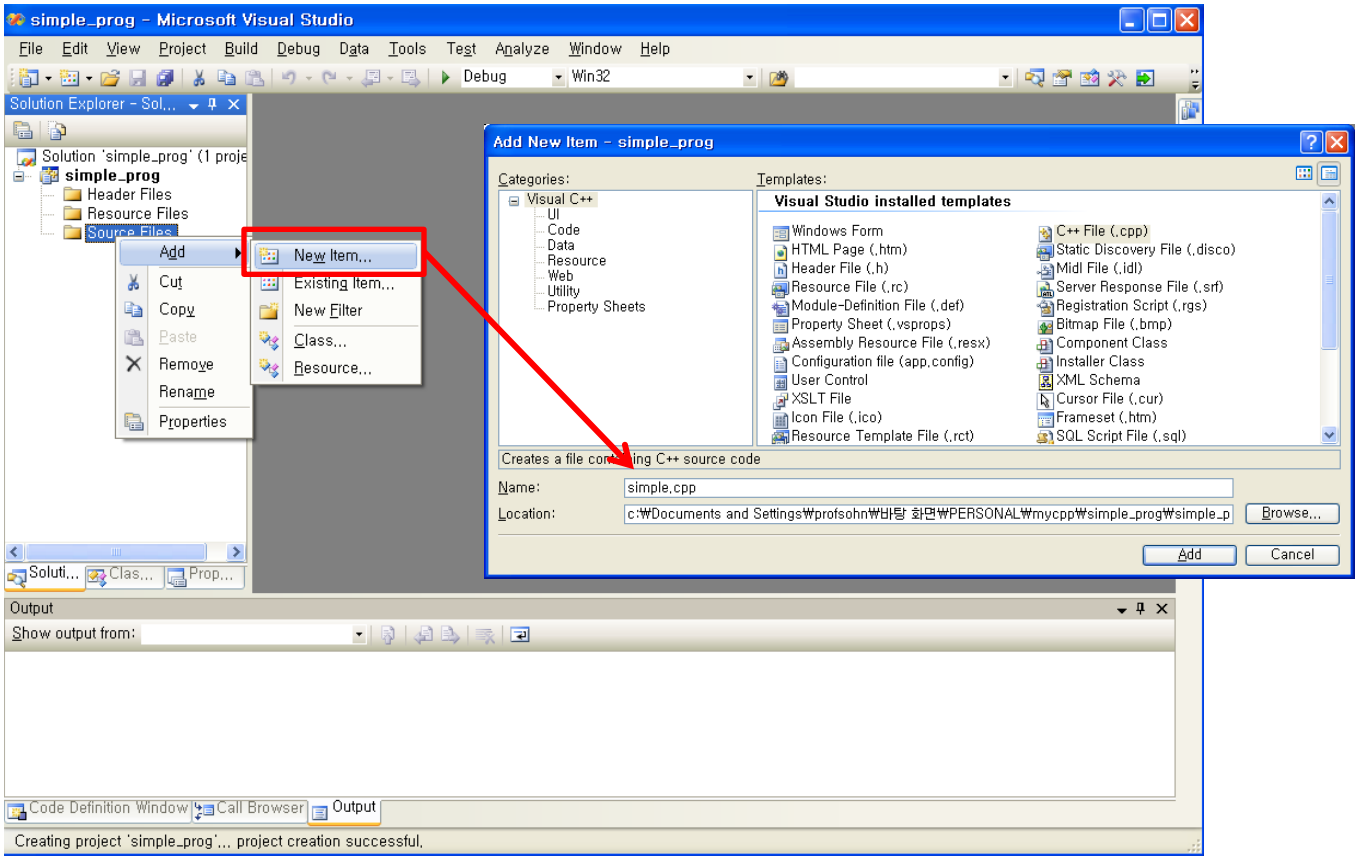


Visual Studio 2008
Creating Command-Line Program

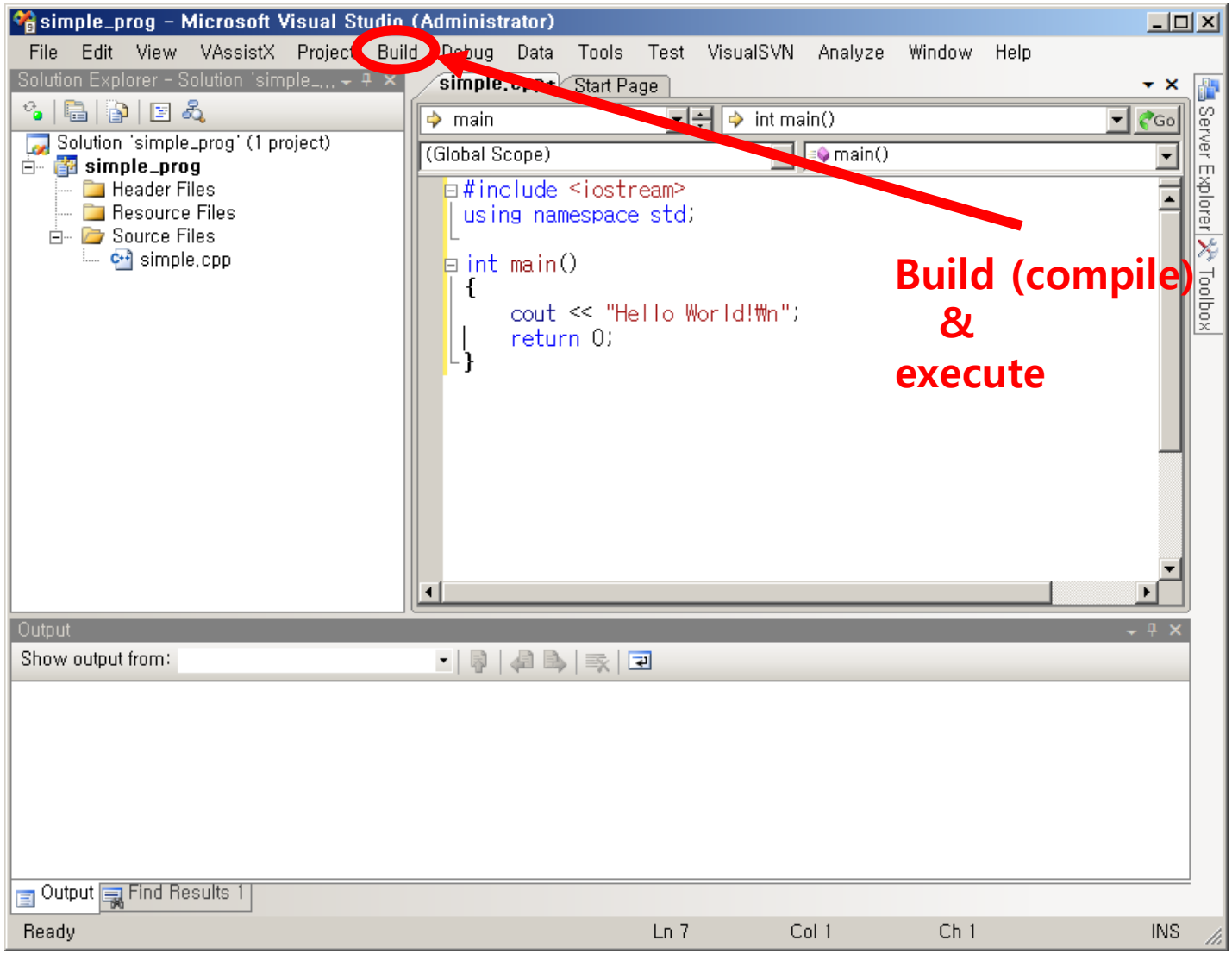




Visual Studio 2008
Creating Command-Line Program



Visual Studio 2008
Creating Command-Line Program



3. “Hello World!” code

```
1 // my second program in C++
2 #include <iostream>
3 using namespace std;
4
5 int main ()
6 {
7   cout << "Hello World! ";
8   cout << "I'm a C++ program";
9 }
```

header file

```
Hello World! I'm a C++ program
```

- It is OK to use C code in C++ program

4. Standard Input/Output

- **`cout`** : screen stream object

  (standard output)

comments

```
1 cout << "Output sentence"; // prints Output sentence on screen
2 cout << 120;               // prints number 120 on screen
3 cout << x;                 // prints the value of x on screen
```

```
1 cout << "Hello";  // prints Hello
2 cout << Hello;    // prints the content of variable Hello
```

```
cout << "I am " << age << " years old and my zipcode is " << zipcode;
```

```
1 cout << "First sentence.₩n";
2 cout << "Second sentence.₩nThird sentence.";
```

4. Standard Input/Output

- **cin** : keyboard stream object (standard input)

```
1 int age;
2 cin >> age;
```

```
1  // i/o example
2
3  #include <iostream>
4  using namespace std;
5
6  int main ()
7  {
8    int i;
9    cout << "Please enter an integer value: ";
10   cin >> i;
11   cout << "The value you entered is " << i;
12   cout << " and its double is " << i*2 << ".₩n";
13   return 0;
14 }
```

```
Please enter an integer value: 702
The value you entered is 702 and its double is 1404.
```

Formatting

- formatting similar to **`printf`**
- **`std::format`** (C++20)
- **`std::print`** (C++23)

```
// src_print.cpp
#include <iostream>
#include <cstdio>
#include <print>
#include <format>

using namespace std;

int main()
{
        double pi=3.141592;
        int age = 21;
        printf("pi=%7.2f , age=%d\n",pi,age);
        cout << format("pi={:7.2f} , age={}\n",pi,age);
        print("pi={:7.2f} , age={}\n",pi,age);
        return 0;
}
```

[program output]

```
pi=   3.14 , age=21
pi=   3.14 , age=21
pi=   3.14 , age=21
```

5. Variables and Data Types

- **Variable**
  - Named memory location where data value is stored
  - Each variable has a certain type
  - Variables must be declared before use in a program
- **`char`** (1 byte) : capable of holding one character (ascii code)
- **`int`** (4 bytes) : integer
- **`float`** (4 bytes) : single-precision floating point
- **`double`** (8 bytes) : double-precision floating point

type	size	min value	max value
`char`	1byte	$-2^7 = -128$	$2^7-1 = 127$
`short`	2byte	$-2^{15} = -32,768$	$2^{15}-1 = 32,767$
`int`	4byte	$-2^{31} = -2,147,483,648$	$2^{31}-1 = 2,147,483,647$

- min/max values are defined in `<limits>` header file
- use **`unsigned`** type when representing only positive numbers
  - **`unsigned char , unsigned short, unsigned int`**

6. `string` type

- standard type for storing and manipulating text
  (a class provided by the C++ standard library)
- Unlike C-style strings (char *),
- **`string`** automatically manages memory and
- provides many convenient functions
- **`getline(cin,str)`** function is used to read a line of text.

- Example code
  - **`src_string.cpp`**
  - **`src_getline.cpp`**

7 : Control Structures
if, loop (while, for)

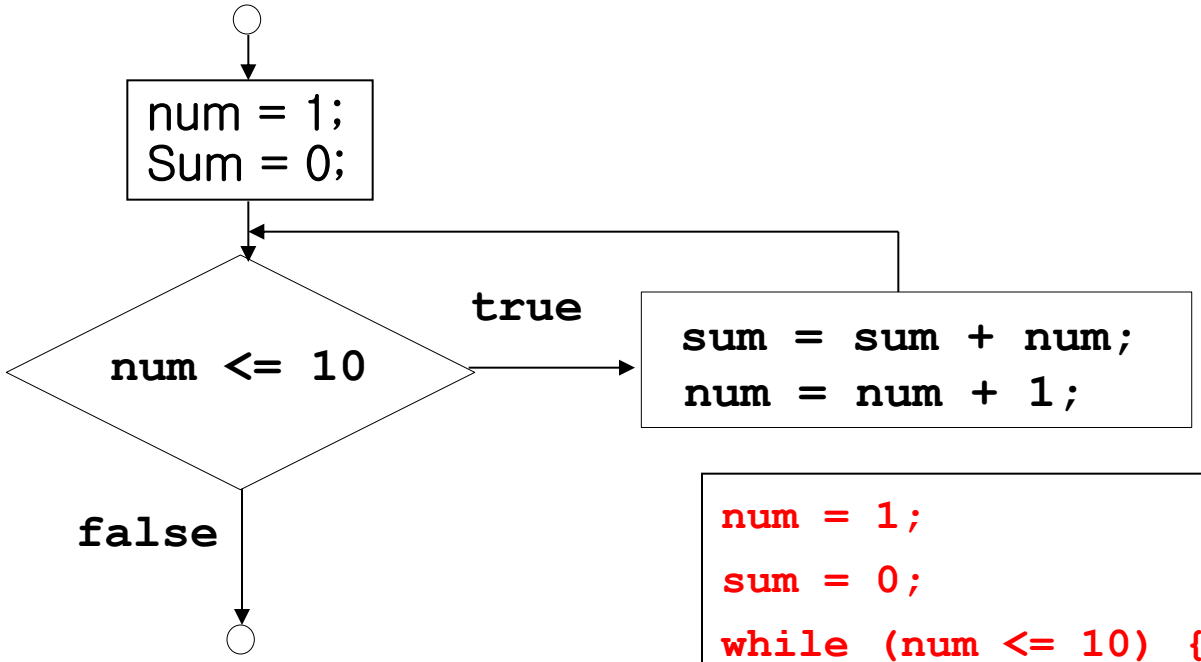
Conditional Statement: if

- block : enclosed by { }
- **if , if-else , if – else if - else**

- Example

```
if ( num1 >= num2 )
{
     printf("num1 is greater than num2\n");
     printf("The difference is: %d\n", num1- num2);
} else {
     printf("num2 is greater than or equal to num1\n";
     printf("The difference is: %d\n", num2 – num1);
}
```

while - loop



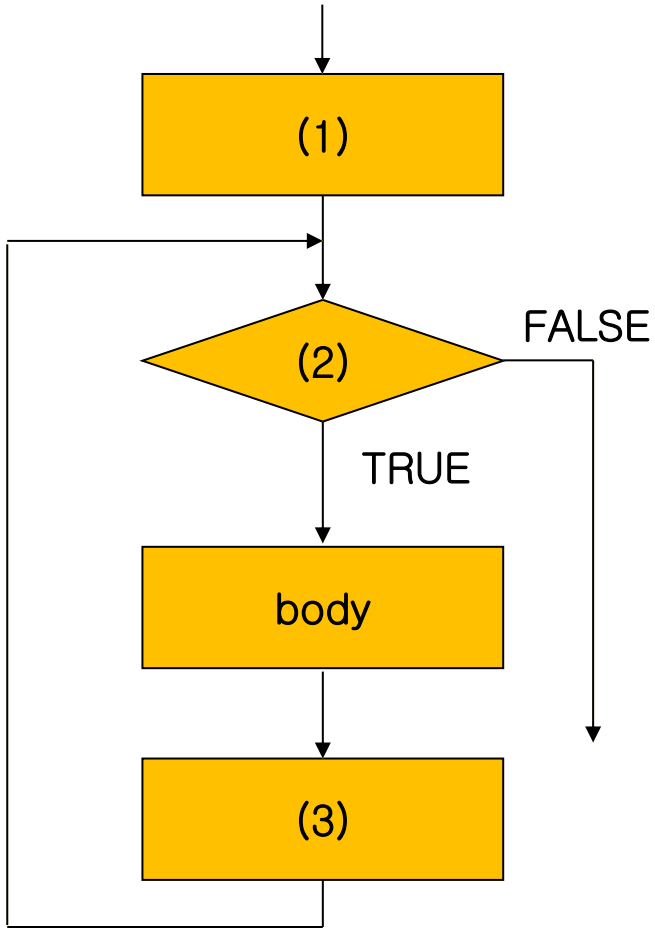

```
num = 1;
sum = 0;
while (num <= 10) {
     sum = sum + num;
     num = num + 1;
 }
```

for - loop

- Repetition

```
for (  (1);    (2);   (3)   )
{
    (body)
}
```

```
(1)  control variable initialization
(2)  Test Conditon
(3)  Modification of control variable value
```



order : (1) (2) body (3) (2) body (3) (2) body ... body (3) (2)

*** Example**

```
sum=0;
for(int num = 1; num <= 10; num++ ) {
     sum += num;
     num++;
}
```

break & continue

- **break** in loop
  - go out of the loop block and execute next to the loop
  - Example

```
while (1) {
    cin >> j;
  if (j == 0)
       break;
    result = i/j;
}
```

- **continue** in loop
  - go to condition-test and next iteration of the loop
  - Example

```
sum = 0;
for (i = 0; i <= n; i++) {
    if (i % 2 == 0)
        continue;
    sum += i;
}
```

8. Functions

```
1  // function example
2  #include <iostream>
3  using namespace std;
4
5  int addition (int a, int b)
6  {
7    int r;
8    r=a+b;
9    return r;
10 }
11
12 int main ()
13 {
14   int z;
15   z = addition (5,3);
16   cout << "The result is " << z;
17 }
```

```
The result is 8
```

- local variables vs global variables
- local variables (including parameter variables)
  - created when function is called
  - deleted when function returns

why use functions?

- many reasons
  - **Don't have to repeat the same block** of code many times. Make that code block a function and call it when needed.
  - **Reuse** : useful functions can be reused in a number of programs.
  - **top-down technique** : Make an outline and hierarchy of the steps needed to solve your problem and create a function for each step.
  - **Easy to debug** : Get one function working well then move on to the others.
  - **Easy to modify and expand** : Just add more functions to extend program capability
  - **Readabilty** : Make program self-documenting and readable.

9. array

- a fixed-sized collection of elements of the same type
- stored consecutively in memory
- fast random access (ex) x[15]=3;
- string as an array of characters (ex) **src_string_cstyle.c**

```
int main()
{
    int x0, x1, x2, x3, x4;

    cout << "Get 5 integers : ";
    cin >> x0;
    cin >> x1;
    cin >> x2;
    cin >> x3;
    cin >> x4;

    cout << "Reverse Order : ";
    cout << x4 << " ";
    cout << x3 << " ";
    cout << x2 << " ";
    cout << x1 << " ";
    cout << x0 << "\n";
    return 0;
}
```



```
int main()
{
    int x[5], i;

    cout << "Get 5 integers : ";
    for (i = 0; i < 5; ++i)
        cin >> x[i];

    cout << "Reverse order : ";
    for (i = 4; i >= 0; --i)
        cout << x[i] << " " ;
    cout << "\n";

    return 0;
}
```



10. pointer

- pointer type variable
  - "pointer" usually means memory address
  - a variable that stores memory address
  - size is always: 8 bytes (in 64-bit systems), 4 bytes (in 32-bit systems)

```
int *xPtr, x;    // xPtr is pointer
                 // x is an integer type

x = 5;           // assignment
xPtr = &x;       // xPtr points to x that means
                 // the value of xPtr is the address of x

*xPtr = 10;      // the value xPtr points to is changed to 10
                 // that means x becomes 10
cout << x;       // result is 10
```

- Remember
  - *****
    - (i) multiplication operator (binary operator), ex) z=x*y;
    - (ii) the-value-of operator (unary operator), ex) *x=5;
    - (iii) pointer type declaration, ex) int *x;
  - **&**
    - (i) bitwise-AND operator (binary operator), ex) z=x&y;
    - (ii) address-of operator (unary operator), ex) x=&y;
    - (iii) reference type declaration, ex) int& y=x;

11. Dynamic memory allocation

- array/pointer : almost the same as C
- dynamic memory allocation
  - "dynamic" means "… occurs at run-time (execution-time)"
  - malloc (in C) ⇔ new (in C++)
  - free (in C) ⇔ delete (in C++)

```
ex) in C
int *x;
x=(int*)malloc(10*sizeof(int));

…

free(x);
```

```
ex) in C++
int *x;
x=new int[10];

…

delete[] x;
```

- static allocation vs dynamic allocation
  - see the class webpage , **`src_array.cpp`**

12. C++ class

class

- class: data structure type that can contain member data and member functions.

```
1  // example: one class, two objects
2  #include <iostream>
3  using namespace std;
4
5  class Rectangle {
6      int width, height;
7    public:
8      void set_values (int,int);
9      int area () {return width*height;}
10 };
11
12 void Rectangle::set_values (int x, int y) {
13   width = x;
14   height = y;
15 }
16
17 int main () {
18   Rectangle rect, rectb;
19   rect.set_values (3,4);
20   rectb.set_values (5,6);
21   cout << "rect area: " << rect.area() << endl;
22   cout << "rectb area: " << rectb.area() << endl;
23   return 0;
24 }
```

```
rect area: 12
rectb area: 30
```

- **class constructor/destructor**

  – member function that is automatically called when the object is created/deleted.

```
1  // example: class constructor
2  #include <iostream>
3  using namespace std;
4
5  class Rectangle {
6      int width, height;
7    public:
8      Rectangle (int,int);
9      int area () {return (width*height);}
10 };
11
12 Rectangle::Rectangle (int a, int b) {
13   width = a;
14   height = b;
15 }
16
17 int main () {
18   Rectangle rect (3,4);
19   Rectangle rectb (5,6);
20   cout << "rect area: " << rect.area() << endl;
21   cout << "rectb area: " << rectb.area() << endl;
22   return 0;
23 }
```

```
rect area: 12
rectb area: 30
```

- **pointers to class object**

```
#include <iostream>
using namespace std;

class Rectangle {
   private:
        int width, height;
   public:
        Rectangle() { width=1; height=1; }
        Rectangle(int x, int y) : width(x), height(y) {}
        int area() { return width*height; }
};

int main()
{
        Rectangle obj(3,4);
        Rectangle *foo, *bar, *baz;

        foo = &obj;
        bar = new Rectangle(5,6);
        baz = new Rectangle[2];

        cout << "obj's area: " << obj.area() << '\n';
        cout << "*foo's area: " << foo->area() << '\n';
        cout << "*bar's area: " << bar->area() << '\n';
        cout << "baz[0]'s area: " << baz[0].area() << '\n';
        cout << "baz[1]'s area: " << baz[1].area() << '\n';
        delete bar;
        delete[] baz;

        return 0;
}
```

13. C++ File Processing

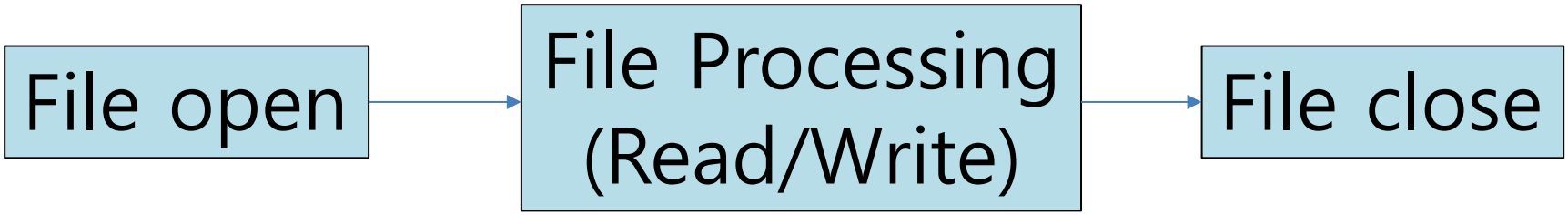
- **file stream class**
  - ofstream: stream class to write on files
  - ifstream: stream class to read from files
  - fstream: stream class to read and write from/to files

```
1  // basic file operations
2  #include <iostream>
3  #include <fstream>
4  using namespace std;
5
6  int main () {
7    ofstream myfile;
8    myfile.open ("example.txt");
9    myfile << "Writing this to a file.₩n";
10   myfile.close();
11   return 0;
12 }
```

```
[file example.txt]
Writing this to a file.
```

File Processing

- File Open
  - Text? Or Binary?
  - In case of writing, append? Or truncate?
  - File Position (automatic? Or User-Control?)



File open mode

The first operation generally performed on an object of one of these classes is to associate it to a real file. This procedure is known as to *open a file*. An open file is represented within a program by a *stream* (i.e., an object of one of these classes; in the previous example, this was `myfile`) and any input or output operation performed on this stream object will be applied to the physical file associated to it.

In order to open a file with a stream object we use its member function `open`:

```
open (filename, mode);
```

Where `filename` is a string representing the name of the file to be opened, and `mode` is an optional parameter with a combination of the following flags:

`ios::in`	Open for input operations.
`ios::out`	Open for output operations.
`ios::binary`	Open in binary mode.
`ios::ate`	Set the initial position at the end of the file. If this flag is not set, the initial position is the beginning of the file.
`ios::app`	All output operations are performed at the end of the file, appending the content to the current content of the file.
`ios::trunc`	If the file is opened for output operations and it already existed, its previous content is deleted and replaced by the new one.

All these flags can be combined using the bitwise operator OR (`|`). For example, if we want to open the file `example.bin` in binary mode to add data we could do it by the following call to member function `open`:

```
1 ofstream myfile;
2 myfile.open ("example.bin", ios::out | ios::app | ios::binary);
```

File Open/Close

- Default mode

class	default mode parameter
ofstream	ios::out
ifstream	ios::in
fstream	ios::in | ios::out

- Check if file open was OK

```
if (myfile.is_open()) { /* ok, proceed with output */ }
```

- Closing a file

```
myfile.close();
```

Text File (Write)- Example

```cpp
// writing on a text file
#include <iostream>
#include <fstream>
using namespace std;

int main () {
  ofstream myfile ("example.txt");
  if (myfile.is_open())
  {
    myfile << "This is a line.\n";
    myfile << "This is another line.\n";
    myfile.close();
  }
  else cout << "Unable to open file";
  return 0;
}
```

```
[file example.txt]
This is a line.
This is another line.
```

Text File (Read) - Example

```
1  // reading a text file
2  #include <iostream>
3  #include <fstream>
4  #include <string>
5  using namespace std;
6
7  int main () {
8    string line;
9    ifstream myfile ("example.txt");
10   if (myfile.is_open())
11   {
12     while ( getline (myfile,line) )
13     {
14       cout << line << '\n';
15     }
16     myfile.close();
17   }
18
19   else cout << "Unable to open file";
20
21   return 0;
22 }
```

```
This is a line.
This is another line.
```

Status Check

Checking state flags

The following member functions exist to check for specific states of a stream (all of them return a `bool` value):

`bad()`
: Returns `true` if a reading or writing operation fails. For example, in the case that we try to write to a file that is not open for writing or if the device where we try to write has no space left.

`fail()`
: Returns `true` in the same cases as `bad()`, but also in the case that a format error happens, like when an alphabetical character is extracted when we are trying to read an integer number.

`eof()`
: Returns `true` if a file open for reading has reached the end.

`good()`
: It is the most generic state flag: it returns `false` in the same cases in which calling any of the previous functions would return `true`. Note that `good` and `bad` are not exact opposites (`good` checks more state flags at once).

The member function `clear()` can be used to reset the state flags.

Get and Put Stream Positioning

- tellg()/tellp()
  - Returns current get/put position

- seekg()/seekp()
  - Change the location of the get/put position

```
seekg(position);
seekp(position);
seekg(offset, direction);
seekp(offset, direction);
```

`ios::beg`	offset counted from the beginning of the stream
`ios::cur`	offset counted from the current position
`ios::end`	offset counted from the end of the stream

File Position Example

```cpp
// obtaining file size
#include <iostream>
#include <fstream>
using namespace std;

int main () {
  streampos begin,end;
  ifstream myfile ("example.bin", ios::binary);
  begin = myfile.tellg();
  myfile.seekg (0, ios::end);
  end = myfile.tellg();
  myfile.close();
  cout << "size is: " << (end-begin) << " bytes.\n";
  return 0;
}
```

```
size is: 40 bytes.
```

Binary File Processing

```
write ( memory_block, size );
read ( memory_block, size );
```

Where `memory_block` is of type `char*` (pointer to `char`), and represents the address of an array of bytes where the read data elements are stored or from where the data elements to be written are taken. The `size` parameter is an integer value that specifies the number of characters to be read or written from/to the memory block.

```
1  // reading an entire binary file
2  #include <iostream>
3  #include <fstream>
4  using namespace std;
5
6  int main () {
7    streampos size;
8    char * memblock;
9
10   ifstream file ("example.bin", ios::in|ios::binary|ios::ate);
11   if (file.is_open())
12   {
13     size = file.tellg();
14     memblock = new char [size];
15     file.seekg (0, ios::beg);
16     file.read (memblock, size);
17     file.close();
18
19     cout << "the entire file content is in memory";
20
21     delete[] memblock;
22   }
23   else cout << "Unable to open file";
24   return 0;
25 }
```

```
the entire file content is in memory
```

보충 자료 (강의 페이지)

- C++ Namespace (cplusplus.com)
  - https://cplusplus.com/doc/oldtutorial/namespaces/
- static vs. dynamic memory allocation (토론 자료)
- 예제 소스: src_print, src_string, src_getline, src_getline_fileio, src_string_cstyle, src_array
  - src_getline_fileio 입력 파일: example.txt

static vs. dynamic memory allocation

[This is not homework. You do not need to submit anything for this.
Just think about the questions below and try answer to the questions.]

static memory allocation vs dynamic memory allocation

when we can create and use arrays.

example of static memory allocation for using array:

```
int x[10];
x[0]=3;
x[2]=7;
```

example of dynamic memory allocation for using array:

```
// C-style
int* x;
x = (int*)malloc(sizeof(x)*10);
x[0]=3;
x[2]=7;
 …
free(x);

// C++ style
int* y;
int n=10;
y = new int[n];
y[0]=3;
y[2]=7;
 …
delete[] y;
```

Question: What are

1. **advantages of the static memory allocation for using array:**
2. **disadvantages of static memory allocation for using array:**
3. **advantages of dynamic memory allocation for using array:**
4. **disadvantages of dynamic memory allocation for using array:**

src_print.cpp

```cpp
#include <iostream>
#include <cstdio>
#include <print>
#include <format>

using namespace std;

int main()
{
    double pi=3.141592;
    int age = 21;
    printf("pi=%7.2f , age=%d\n",pi,age);
    cout << format("pi={:7.2f} , age={}\n",pi,age);
    print("pi={:7.2f} , age={}\n",pi,age);

    return 0;
}
```

src_string.cpp (1/4)

```cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    // 1. Creating strings
    string s1 = "Hello";
    string s2("World");
    string s3;                  // empty string

    // 2. Concatenation
    s3 = s1 + " " + s2;
    cout << s3 << endl;         // Hello World

    // 3. Length
    cout << "Length: " << s3.length() << endl;
    cout << "Size: " << s3.size() << endl;  // same as length()

    // 4. Accessing individual characters
    cout << s3[0] << endl;      // H
    cout << s3.at(1) << endl;   // e

    // 5. Modifying a character
    s3[0] = 'h';
    cout << s3 << endl;         // hello World

    // 6. Adding text
```

src_string.cpp (2/4)

```
28      s3 += "!";
29      s3.append(" C++");
30      cout << s3 << endl;         // hello World! C++
31
32      // 7. Finding a substring
33      size_t pos = s3.find("World");
34
35      if (pos != string::npos)
36          cout << "Found at index: " << pos << endl;
37
38      // 8. Extracting a substring
39      string sub = s3.substr(6, 5);
40      cout << sub << endl;        // World
41
42      // 9. Replacing part of a string
43      s3.replace(6, 5, "C++");
44      cout << s3 << endl;         // hello C++! C++
45
46      // 10. Inserting text
47      s3.insert(5, " beautiful");
48      cout << s3 << endl;         // hello beautiful C++! C++
49
50      // 11. Erasing part of a string
51      s3.erase(5, 10);
52      cout << s3 << endl;         // hello C++! C++
53
54      // 12. String comparison
```

src_string.cpp (3/4)

```
55      string a = "apple";
56      string b = "banana";
57
58      if (a < b)
59          cout << "apple comes before banana" << endl;
60
61      if (a == "apple")
62          cout << "a is apple" << endl;
63
64      // 13. Checking whether a string is empty
65      if (s3.empty())
66          cout << "String is empty" << endl;
67
68      // 14. Iterating through characters
69      for (char c : s3)
70          cout << c << ' ';
71      cout << endl;
72
73      // 15. C-style string when needed
74      const char* p = s3.c_str();
75      cout << p << endl;
76
77      // 16. Numeric Conversions
78      string doubleStr = "123.45";
79      string intStr = "67";
80      double dval = stod(doubleStr);   // String to double
81      int ival = stoi(intStr);   // String to int
```

src_string.cpp (4/4)

```
82      string formatted1 = to_string(dval * 2); // Number to string
83      string formatted2 = to_string(ival * 2); // Number to string
84      cout << "conversion result: " << formatted1 << " , " << formatted2 << "\n";
85
86      return 0;
87  }
```

src_getline.cpp (1/2)

```
 1  #include <iostream>
 2  #include <string>
 3  #include <sstream>
 4  #include <fstream>
 5  using namespace std;
 6
 7  int main()
 8  {
 9      cout << "[student_id] [name] [department] [age] [GPA(score)]\n";
10
11      string student_id;
12      string name;
13      string dept;
14      int age;
15      double GPA;
16      double GPA_total=0.0;
17      string linestr;
18      int count=0;
19
20
21      while (getline(cin,linestr)) {
22  //  ifstream myfile("example.txt");
23  //  while (getline(myfile,linestr)) {
24          if (linestr.empty()) break;
25          istringstream iss(linestr); // istringstream treats a string as an input
stream.
26          iss >> student_id >> name >> dept >> age >> GPA ;
27          cout << "student_id: " << student_id << "\n";
```

src_getline.cpp (2/2)

```
28          cout << "name: " << name << "\n";
29          cout << "department: " << dept << "\n";
30          cout << "age: " << age << "\n";
31          cout << "GPA(score): " << GPA << "\n";
32          GPA_total += GPA;
33          count++;
34      }
35  //  myfile.close();
36      cout << "GPA average: " << GPA_total/count << "\n";
37
38      return 0;
39  }
```

src_getline_fileio.cpp (1/2)

```
1  #include <iostream>
2  #include <string>
3  #include <sstream>
4  #include <fstream>
5  using namespace std;
6
7  int main()
8  {
9      cout << "[student_id] [name] [department] [age] [GPA(score)]\n";
10
11     string student_id;
12     string name;
13     string dept;
14     int age;
15     double GPA;
16     double GPA_total=0.0;
17     string linestr;
18     string temp_str;
19     int count=0;
20
21
22 //    while (getline(cin,linestr)) {
23   ifstream myfile("example.txt");
24   ofstream out("example_out.txt");
25   while (getline(myfile,linestr)) {
26         if (linestr.empty()) break;
27
```

src_getline_fileio.cpp (2/2)

```
28          student_id=linestr.substr(0,8);
29          name=linestr.substr(10,10);
30          dept=linestr.substr(20,10);
31          age=stoi(linestr.substr(30,3));
32          GPA=stod(linestr.substr(40,4));
33
34          cout << "student_id: " << student_id << "\n";
35          cout << "name: " << name << "\n";
36          cout << "department: " << dept << "\n";
37          cout << "age: " << age << "\n";
38          cout << "GPA(score): " << GPA << "\n";
39          GPA_total += GPA;
40          count++;
41          temp_str=format("{:10s}{:20s}{:20s}{:10d}{:10.2f}\n",student_id,name,dept
,age,GPA);
42          cout << temp_str;
43          out << temp_str;
44      }
45    myfile.close();
46    out.close();
47      cout << "GPA average: " << GPA_total/count << "\n";
48
49      return 0;
50  }
```

example.txt (src_getline_fileio 입력)

```
1  20221523  Lisa_Kim  Comp_Sci  21        4.12
2  20242123  Homer_Lee Elec_Eng  22        3.51
3  20231118  Bart_Park Mech_Eng  23        3.81
```

src_string_cstyle.c (1/4)

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main(void) {
    // 1. Creating strings
    char s1[100] = "Hello";
    char s2[100] = "World";
    char s3[200] = "";              // empty string

    // 2. Concatenation
    strcpy(s3, s1);
    strcat(s3, " ");
    strcat(s3, s2);
    printf("%s\n", s3);             // Hello World

    // 3. Length
    printf("Length: %zu\n", strlen(s3));

    // 4. Accessing individual characters
    printf("%c\n", s3[0]);          // H
    printf("%c\n", s3[1]);          // e

    // 5. Modifying a character
    s3[0] = 'h';
    printf("%s\n", s3);             // hello World

```

src_string_cstyle.c (2/4)

```
28      // 6. Adding text
29      strcat(s3, "!");
30      strcat(s3, " C");
31      printf("%s\n", s3);             // hello World! C
32
33      // 7. Finding a substring
34      char *pos = strstr(s3, "World");
35
36      if (pos != NULL)
37          printf("Found at index: %td\n", pos - s3);
38
39      // 8. Extracting a substring
40      char sub[6];
41      strncpy(sub, s3 + 6, 5);
42      sub[5] = '\0';
43      printf("%s\n", sub);            // World
44
45      // 9. Replacing part of a string
46      // replacement is not supported in C
47
48      // 10. Inserting text
49      // insertion is not supported in C
50
51      // 11. Erasing part of a string
52      // erase 6 characters starting at index 5
53      // erase is not supported
54
```

src_string_cstyle.c (3/4)

```
55      // 12. String comparison
56      char a[] = "apple";
57      char b[] = "banana";
58
59      if (strcmp(a, b) < 0)
60          printf("apple comes before banana\n");
61
62      if (strcmp(a, "apple") == 0)
63          printf("a is apple\n");
64
65      // 13. Checking whether a string is empty
66      if (strlen(s3) == 0)
67          printf("String is empty\n");
68
69      // 14. Iterating through characters
70      for (size_t i = 0; i < strlen(s3); i++)
71          printf("%c ", s3[i]);
72
73      printf("\n");
74
75      // 15. C-style string
76      const char *p = s3;
77      printf("%s\n", p);
78
79      // 16. Numeric conversions
80      char doubleStr[] = "123.45";
81      char intStr[] = "67";
```

src_string_cstyle.c (4/4)

```
82
83      double dval = atof(doubleStr);   // String to double
84      int ival = (int)atoi(intStr); // String to int
85
86      char formatted1[50];
87      char formatted2[50];
88
89      sprintf(formatted1, "%f", dval * 2);
90      sprintf(formatted2, "%d", ival * 2);
91
92      printf("conversion result: %s , %s\n",
93             formatted1, formatted2);
94
95      return 0;
96  }
```

src_array.cpp (1/2)

```cpp
#include <iostream>
using namespace std;

int main() {
    // 1. Standard Array
    int stackArray[3] = {10, 20, 30};

    // 2. Dynamic Memory Allocation
    int size;
    cout << "input size: ";
    cin >> size;
    int* heapArray = new int[size];

    // 3. Pointer Operations
    int* ptrToStack = stackArray;

    // Populate the heap array using pointer offset notation
    cout << "enter " << size << " integers: \n";
    for (int i = 0; i < size; ++i) {
        cin >> heapArray[i];
    }

    // 4. Output Comparisons
    for (int i = 0; i < 3; ++i) {
        cout << "stackArray[" << i << "]=" << stackArray[i]
             << " | *(ptrToStack+" << i << ")=" << *(ptrToStack + i) << "\n";
    }
```

src_array.cpp (2/2)

```
28
29      for (int i = 0; i < size; ++i) {
30          cout << "*(heapArray+" << i << ")= " << *(heapArray + i)
31                    << " | heapArray[" << i << "]=" << heapArray[i] << "\n";
32      }
33
34      // 5. Memory Cleanup
35      // Only heap-allocated memory requires manual deletion.
36      delete[] heapArray;
37
38      return 0;
39  }
```